# 1.yes,2.A public demo site,3/ both, 4/Foundry Local model, 5/ I need only local proof of concept.Don't require GitHub Actions.

Approved. Since the proof of concept must run on **your Windows laptop**, I'll give you the exact local setup and ready-to-run files. Foundry Local runs models on-device and can expose an OpenAI-compatible local REST endpoint, so Python can call your LLM/VLM without cloud services.[1] [2]

### Phase 1: Check Foundry Local

Open **PowerShell** and run:

```
foundry --version
foundry model list
```

If `foundry` is not recognized, reinstall or repair Foundry Local, then reopen PowerShell.

To see models that can run on your hardware, use:

```
foundry model list --hardware
```

Choose:

- **LLM:** a small model such as `qwen2.5-0.5b` for fast test-generation experiments.
- **VLM:** a vision model such as `qwen3.5-vision`, if it appears in your catalog. Foundry Local 1.1 added Qwen 3.5 Vision for multimodal image-and-text input.[3] [4]

Download the models you select:

```
foundry model download qwen2.5-0.5b
foundry model download qwen3.5-vision
```

If either alias is unavailable, copy the exact model alias shown by `foundry model list` and use that instead. The official quickstart notes that model aliases can vary by device/catalog.[1]

### Phase 2: Create the project

In PowerShell:

```
mkdir ai-regression-tests
cd ai-regression-tests
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install pytest pytest-playwright openai foundry-local-sdk-winml
playwright install
```

If your Foundry Local installation uses the cross-platform package instead, replace `foundry-local-sdk-winml` with `foundry-local-sdk`. Microsoft's Python examples use these packages for the Foundry Local SDK and OpenAI-compatible client.[2] [1]

### Phase 3: Create the Foundry client

Create `foundry_client.py`:

```
import base64
import json
import re
from pathlib import Path

from foundry_local_sdk import Configuration, FoundryLocalManager
from openai import OpenAI

LLM_ALIAS = "qwen2.5-0.5b"
VLM_ALIAS = "qwen3.5-vision"


def get_client_and_models():
    config = Configuration(app_name="ai_regression_tests")
    FoundryLocalManager.initialize(config)
    manager = FoundryLocalManager.instance

    manager.download_and_register_eps()

    llm = manager.catalog.get_model(LLM_ALIAS)
    vlm = manager.catalog.get_model(VLM_ALIAS)

    if llm is None:
        raise RuntimeError(f"LLM model not found: {LLM_ALIAS}")
    if vlm is None:
        raise RuntimeError(f"VLM model not found: {VLM_ALIAS}")

    for model in (llm, vlm):
        if not model.is_cached:
            model.download(lambda progress: print(
                f"Downloading {model.alias}: {progress:.1f}%",
                end="", flush=True
            ))
            print()
        model.load()
```

```
    manager.start_web_service()
    base_url = f"{manager.urls[^0]}/v1"

    client = OpenAI(base_url=base_url, api_key="notneeded")
    return client, llm.id, vlm.id, manager, llm, vlm


def generate_test_cases(client, model_id, feature_description):
    prompt = f"""
Generate regression test cases for this feature:

{feature_description}

Return only valid JSON in this format:
{{
  "test_cases": [
    {{
      "title": "",
      "priority": "high|medium|low",
      "steps": [],
      "test_data": {{}},
      "expected_result": ""
    }}
  ]
}}
"""

    response = client.chat.completions.create(
        model=model_id,
        messages=[
            {"role": "system", "content": "You are a precise QA test designer."},
            {"role": "user", "content": prompt},
        ],
        temperature=0.2,
    )

    text = response.choices[^0].message.content
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        raise ValueError(f"LLM did not return JSON:\n{text}")

    return json.loads(match.group())


def analyze_screenshot(client, model_id, screenshot_path, expected_rules):
    image_b64 = base64.b64encode(Path(screenshot_path).read_bytes()).decode()

    prompt = f"""
You are reviewing a web-page screenshot for regression testing.

Expected rules:
{expected_rules}

Return only valid JSON:
{{
```

```
  "status": "pass|fail",
  "issues": [],
  "summary": ""
}}

Only report visible, factual UI issues. Do not invent issues.
"""

    response = client.chat.completions.create(
        model=model_id,
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"{image_b64}"
                        },
                    },
                ],
            }
        ],
        temperature=0.0,
    )

    text = response.choices[^0].message.content
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        raise ValueError(f"VLM did not return JSON:\n{text}")

    return json.loads(match.group())
```

This uses Foundry Local's OpenAI-compatible `/v1` endpoint; the official integration guide shows the same pattern of starting the local web service and connecting the OpenAI SDK to it.[2]

### Phase 4: Create the regression test

Create `tests/test_ai_regression.py`:

```
import json
from pathlib import Path

import pytest
from playwright.sync_api import Page, expect

from foundry_client import analyze_screenshot, generate_test_cases, get_client_and_m

SCREENSHOTS = Path("screenshots")
SCREENSHOTS.mkdir(exist_ok=True)

TARGET_URL = "https://playwright.dev/"
```

```
@pytest.fixture(scope="session")
def foundry():
    client, llm_id, vlm_id, manager, llm, vlm = get_client_and_models()
    yield {
        "client": client,
        "llm_id": llm_id,
        "vlm_id": vlm_id,
        "manager": manager,
        "llm": llm,
        "vlm": vlm,
    }
    manager.stop_web_service()
    for model in (llm, vlm):
        if model.is_loaded:
            model.unload()
    manager.close()


def test_llm_generates_login_regression_cases(foundry):
    result = generate_test_cases(
        foundry["client"],
        foundry["llm_id"],
        "A login page with email, password, sign-in button, error messages, and forgo
    )

    assert "test_cases" in result
    assert len(result["test_cases"]) > 0

    output = {"llm_model": foundry["llm_id"], "result": result}
    Path("reports").mkdir(exist_ok=True)
    Path("reports/llm_test_cases.json").write_text(
        json.dumps(output, indent=2), encoding="utf-8"
    )


def test_vlm_checks_playwright_homepage(page: Page, foundry):
    page.goto(TARGET_URL)
    page.screenshot(path=str(SCREENSHOTS / "home.png"), full_page=True)

    result = analyze_screenshot(
        foundry["client"],
        foundry["vlm_id"],
        SCREENSHOTS / "home.png",
        """
        - The page must load with visible content.
        - The page must not show a browser error page.
        - The page must contain navigation or documentation links.
        """,
    )

    Path("reports").mkdir(exist_ok=True)
    Path("reports/vlm_home_review.json").write_text(
        json.dumps(result, indent=2), encoding="utf-8"
    )
```

```
    print("\nVLM result:", json.dumps(result, indent=2))

    assert result.get("status") in ("pass", "fail")
    if result.get("status") == "fail":
        assert result.get("issues"), "VLM reported failure without listing issues."
```

### Phase 5: Run the proof of concept

From the project folder, with the virtual environment active:

```
pytest -s -v
```

You should get:

- `reports/llm_test_cases.json`: AI-generated regression test cases.
- `screenshots/home.png`: Browser screenshot captured by Playwright.
- `reports/vlm_home_review.json`: VLM's structured review of the screenshot.
- Terminal output showing whether each test passed.

### Important design rule

Use the **VLM as a reviewer**, not as the only pass/fail authority. Keep deterministic Playwright assertions for critical functionality, and use the VLM to surface visual or contextual issues that scripted checks may miss. This reduces false failures from subjective AI judgments. [5] [6] [3]

### If a model alias fails

Run:

```
foundry model list
```

Then update these two lines in `foundry_client.py` with the exact aliases shown:

```
LLM_ALIAS = "your-text-model-alias"
VLM_ALIAS = "your-vision-model-alias"
```

If no VLM is available in your current Foundry Local catalog, we can still complete the LLM test-generation proof of concept and add VLM screenshot analysis after you install or enable a supported vision model. [4] [3] [1]

Run the Phase 1 commands and paste the output of:

```
foundry --version
foundry model list
```

I'll then tell you the exact model aliases to put in `foundry_client.py` and troubleshoot any installation or model-loading errors.

⁂


1. https://learn.microsoft.com/en-us/azure/foundry-local/get-started
2. https://learn.microsoft.com/en-us/azure/foundry-local/how-to/how-to-integrate-with-inference-sdks
3. https://devblogs.microsoft.com/foundry/foundry-local-v1-1/
4. https://devblogs.microsoft.com/foundry/whats-new-in-microsoft-foundry-may-2026/
5. https://keploy.io/blog/community/automated-regression-testing
6. https://www.devopssupport.in/blog/the-complete-guide-to-automated-testing-in-ci-cd-pipelines/